AN EXPERIMENTAL ANALYSIS OF MONGODB AND POSTGRESQL PERFORMANCE UNDER VARYING DATA VOLUMES, QUERY COMPLEXITIES, AND INDEXING STRATEGIES

Dharaneesh S L[1] **Dr. Saranya K G**[2]

[1]Student, Department of Computer Science and Engineering, PSG College of Technology, Coimbatore, Tamil Nadu, India
[2]Associate Professor, Department of Computer Science and Engineering, PSG College of Technology, Coimbatore, Tamil Nadu, India

[1]dharaneesh19112006@gmail.com [2]kgs.cse@psgtech.ac.in

Abstract: Problem: Selecting a relational or document database requires measurements that distinguish schema choices from deployment and execution costs. **Objective:** This study evaluates PostgreSQL and MongoDB across the planned scale factors 0.1, 0.5, and 1.0, three core query workloads, and two secondary-index configurations. **MongoDB vs. PostgreSQL:** Both systems use the same eight-entity, TPC-H-derived logical schema. **Experimental Methodology:** Each completed configuration uses three warm-up runs and ten measured runs. Client-side latency is recorded through full result retrieval, executed plans are retained, and output-matched comparisons use a two-sided Mann–Whitney U test with Holm adjustment. A 60-second server-side query limit applies. **Main Performance Metrics:** The evidence records latency, batch CRUD throughput, allocated data and index storage, and execution-plan counters. Local server-process CPU and aggregate RSS were sampled with the stated accounting limitations. **Key Findings:** PostgreSQL had the lower observed median in 10 of the 10 complete core-query comparisons, and MongoDB in 0. Completed-comparison speedups ranged from 1.83× to 232.46×. **Conclusion:** Both database servers ran on the same Windows host over loopback connections. The results characterize the recorded single-node settings, derived datasets, and warm-cache workloads. Workload-specific medians and indexing effects provide the basis for the reported comparisons. Incomplete or timed-out conditions are identified explicitly.

Index terms: MongoDB, PostgreSQL, NoSQL, relational database, TPC-H, benchmarking, indexing, query performance, execution plans, empirical evaluation.

I. INTRODUCTION

A. Background

The volume of data that applications must store and query has influenced the design of database systems [1]. A single platform may need to ingest both structured transaction records and semi-structured event logs at high rates. The choice of storage engine depends on how well it can ingest and index those records and serve the queries the application actually issues [2].

B. Relational Databases

Relational database management systems represent data in tables [3]. PostgreSQL supports normalized schemas, SQL queries, foreign-key constraints, and ACID transactions. Its cost-based optimizer uses table statistics and assumptions about available memory to choose among physical operators, including sequential scans, index scans, and hash joins [4, 5].

C. NoSQL and MongoDB

MongoDB stores self-describing BSON documents in collections with flexible schemas [6]. Each document can contain a hierarchy of nested values. Sharding distributes data horizontally across machines. For the join workloads examined here, MongoDB expresses the query as a sequence of aggregation stages, including `$lookup` for access to a related collection. PostgreSQL expresses the corresponding join through its cost-based relational planner [7, 8].

D. Problem Statement

Despite widespread use of both systems, published comparisons give limited guidance on when MongoDB will outperform a relational database or when the reverse will hold. If a benchmark embeds related records in MongoDB and places them in separate relational tables, the measured difference reflects both the engine and the decision to denormalize.

E. Research Motivation

To isolate the contribution of the engine, we held the logical schema constant. Varying data volume, query complexity, and indexing together allowed us to identify the workload conditions under which the performance advantage shifted between systems.

F. Research Objective

Our objective was to build a reproducible benchmark for measuring latency, throughput, and resource use in MongoDB and PostgreSQL under the same logical conditions, using equivalent datasets derived from TPC-H.

G. Research Questions

- **RQ1:** How does data volume affect create, read, update, and delete (CRUD) performance in MongoDB and PostgreSQL?
- **RQ2:** How does query complexity affect response time in each system?
- **RQ3:** How do secondary indexes affect read and write performance?
- **RQ4:** How do the two systems compare in storage, CPU, and memory requirements?
- **RQ5:** Which workload characteristics favor MongoDB, and which favor PostgreSQL?

II. LITERATURE REVIEW

Previous Research: Comparisons of relational and NoSQL systems have become more common over the past decade. Cattell [9] surveyed scalable data stores, and Stonebraker and colleagues [10] argued that different workloads call for different database architectures. Sadalage and Fowler [11] helped popularize polyglot persistence, the practice of using different stores for different parts of an application. Leavitt [12] and Mohamed *et al.* [13] examined the trade-offs involved in adopting NoSQL. Han *et al.* [14] and Gessert *et al.* [15] classified systems by data model and consistency properties.

Empirical studies have compared MongoDB with MySQL, SQL Server, and PostgreSQL, respectively, in work by Győrödi *et al.* [16], Parker *et al.* [17], and Jung *et al.* [18]. Boicea *et al.* [19] compared MongoDB with Oracle. Li and Manoharan [20] examined mixed SQL and NoSQL workloads, while Abramova and Bernardino [21] studied MongoDB and Cassandra. YCSB [22] and OLTP-Bench [23] provide frameworks for benchmarking. Raasveldt *et al.* [24] discuss the experimental controls needed for fair comparisons.

A. Research Gap

Earlier comparisons typically vary data volume for a simple CRUD workload or vary query complexity at a fixed volume. Fewer studies relate client-side latency to specific plan measurements, such as buffer hits and documents examined, while also testing the differences between engines with a nonparametric method.

B. Hypotheses

- **H1:** Increasing dataset size increases execution time for both databases [16, 25].
- **H2:** Increasing query complexity increases response time [26].
- **H3:** Appropriate indexing substantially reduces query response time [27].
- **H4:** Indexing introduces additional storage and write-maintenance overhead.
- **H5:** MongoDB and PostgreSQL exhibit meaningfully different performance profiles across workload types [1].
- **H6 (Expectation):** We expect PostgreSQL to perform better on analytical workloads that require multi-way joins and MongoDB to perform better on simple queries that access a single document [15].

III. DATASET AND EXPERIMENTAL ENVIRONMENT

A. TPC-H Dataset

The datasets follow the repository's TPC-H-derived generator [28, 29] at the planned scale factors SF $\in \{0.1, 0.5, 1.0\}$. SF 0.1 was exported from the existing Atlas data. Larger datasets were generated with the repository's deterministic seeds. These are derived data distributions and locally defined query identifiers. Each dataset contains all eight entities: `CUSTOMER`, `ORDERS`, `LINEITEM`, `PART`, `SUPPLIER`, `PARTSUPP`, `NATION`, and `REGION`.

B. Hardware Configuration

Table 1 lists the verified client and PostgreSQL host environment and the database versions returned by the live servers. Both servers and the client run on this host. The initial Atlas dataset was exported before the local comparison.

Table 1
Experimental Environment (Hardware and Software)

Component	Value
CPU	Intel Core i7-14650HX
RAM	23.64 GiB
Storage	Samsung 1 TB SSD
Operating System	Windows 11
MongoDB Version	8.0.37 (local)
PostgreSQL Version	18.7 (local)
Python Version	3.14.7
PyMongo driver	4.19.0
Psycopg2 driver	2.9.13

C. Software Stack

The Python client uses PyMongo 4.19.0 and Psycopg2 2.9.13. Local MongoDB 8.0.37 uses WiredTiger and PostgreSQL reports version 18.7. PostgreSQL uses synchronous commit. MongoDB writes use acknowledged, journaled writes ($w = 1$, journal=true). PostgreSQL uses shared_buffers=128 MiB, work_mem=4 MiB, and up to two parallel workers per gather. WiredTiger's configured maximum cache is 1,073,741,824 bytes (1 GiB). The configured settings therefore imply different cache budgets. Analysis uses pandas 3.0.6, NumPy 2.5.3, and SciPy 1.18.1.

IV. PROPOSED FRAMEWORK

A. Architecture

Figure 1 traces a benchmark run through five layers. The TPC-H data generator writes CSV files for each scale factor, and the data loader imports them into the two databases. The Python benchmark engine runs the queries, switches the index configuration, and records the measurements. The statistical analysis layer reads the resulting CSV files and applies the Mann–Whitney U test.

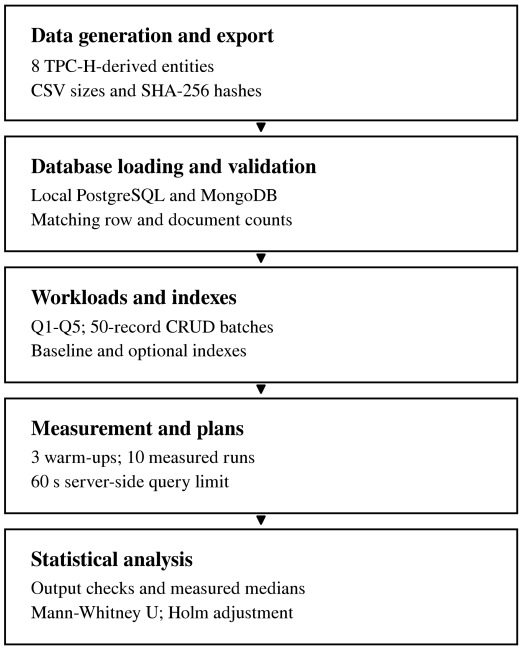


Figure 1: Five layers of the benchmark architecture. Data generation and loading precede query execution and measurement. Statistical analysis uses the recorded output.

B. MongoDB Implementation

We created one collection for each of the eight TPC-H entities. Keeping related records in separate collections preserved the logical schema and removed denormalization as a source of variation in the comparison.

C. PostgreSQL Architecture

The PostgreSQL implementation uses eight normalized tables with declared primary and foreign keys. MongoDB stores corresponding entities in separate collections. Both baseline modes retain mandatory identifier indexes. In particular, PostgreSQL retains its primary-key indexes, whereas MongoDB retains its `_id` indexes. The indexed mode adds the seven workload indexes defined in the repository on matching predicate and join fields. The baseline therefore means no optional workload indexes, rather than a complete absence of indexes.

D. Data Consistency

Figure 2 shows how the same entities were related in each database. In PostgreSQL, foreign keys enforce referential integrity along the `CUSTOMER` → `ORDERS` → `LINEITEM` chain. MongoDB links the corresponding collections when a query runs. Before taking measurements, we compared the row count in each table and the document count in each collection with the count in the source CSV file.

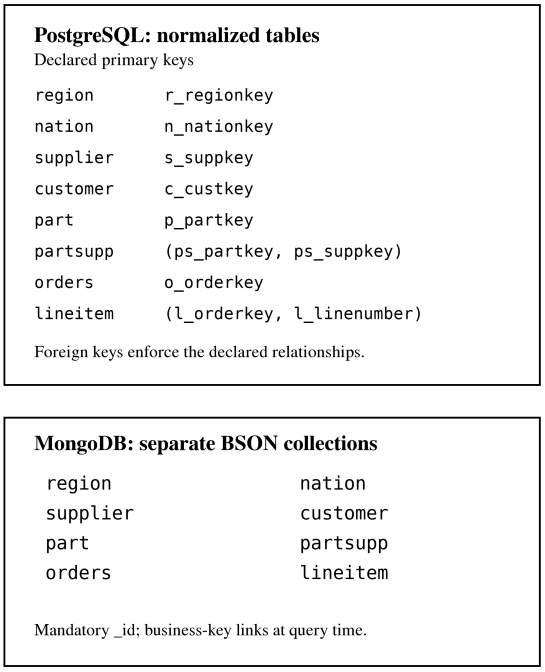


Figure 2: Logical schema in PostgreSQL and MongoDB. Each entity has a corresponding table and collection, so the comparison holds the choice of schema constant.

V. EXPERIMENTAL METHODOLOGY

A. CRUD Experiments

Each completed configuration has three unmeasured warm-up runs followed by ten measured runs [24]. Engine order alternates between runs. CRUD trials operate on batches of 50 records. PostgreSQL bulk inserts use one `execute_values` statement and MongoDB uses `insert_many`. Reads issue 50 successive key lookups. Updates and deletes affect 50 records. Disposable insert/delete batches are restored before each trial, and updates start from the same account balance. Query timing ends after full result retrieval and client materialization. PostgreSQL rows are mapped to dictionaries, matching the MongoDB result representation. Canonicalization and multiset validation occur outside the timed interval. Both query/index configurations are measured before CRUD mutations. A server-side 60-second limit applies to each query. A timeout stops that engine's remaining repetitions for that configuration and is reported as TO. Conditions without complete measurements are reported as NM.

B. Query Experiments

Table 2 groups the queries by structural complexity and identifies the physical operators expected for each engine.

Table 2
Workload complexity and expected execution operators

Workload	Query Type	Complexity Tier	MongoDB Expected Operator	PostgreSQL Expected Operator
$Q1$	Equality Lookup	Low	`IXSCAN → FETCH`	`Index Scan`
$Q4$	Two-way Join	Moderate	`IXSCAN → $lookup`	`Hash Join (Index Scan)`
$Q5$	Join + Aggregation	High	`$lookup → $match → $group`	`Hash Join + HashAggregate`

*1) Simple Queries: Q*1 (*Equality lookup*) retrieves the customer with `c_custkey=100`. PostgreSQL declares this field as a primary key. In MongoDB it is a business-key field distinct from `_id`. It represents the low-complexity tier.

*2) Moderate Queries: Q*4 (*Two-way join*) joins `CUSTOMER` to `ORDERS` on an equality condition and represents the moderate-complexity tier.

*3) Complex Queries: Q*5 (*Join + aggregation*) represents the high-complexity tier. It joins `ORDERS` and `LINEITEM` on the order key, filters ship dates from 1994-01-01 inclusive to 1995-01-01 exclusive, groups by order priority, computes a count and the sum of extended price multiplied by one minus discount, and sorts by revenue. The implementation contains one join between two entities. MongoDB uses numeric doubles and PostgreSQL uses exact numeric values. Aggregate output validation rounds revenue to four decimal places.

VI. PERFORMANCE MEASUREMENTS

A. Insert Throughput

We calculated throughput for each measured 50-record insert batch as

$$\text{Throughput} = \frac{N_{\text{ops}}}{T_{\text{total}}} \tag{1}$$

where $N_{\text{ops}} = 50$ is the number of acknowledged records and T_{total} is the batch wall-clock time. Figure 3 reports the observed medians. The comparison uses loopback connections and each deployment's stated write acknowledgment. At SF 1.0 with optional indexes, the median insert-batch latency was 5.68 ms for MongoDB and 2.84 ms for PostgreSQL. Their respective throughputs were 8,817 and 17,591 records/s, corresponding to a reported 2.00× PostgreSQL speedup.

B. Update Performance

Update trials add 10.00 to the account balances of 50 customers after restoring their starting values. The measured batch medians are reported in Figure 4. The measurements determine the scaling pattern. A linear relation is not assumed. For indexed SF 1.0, median update-batch latency was 3.40 ms for MongoDB and 0.84 ms for PostgreSQL, with respective throughputs of 14,725 and 59,690 records/s and a reported 4.05× PostgreSQL speedup.

C. Delete Performance

Each delete trial removes a freshly restored disposable batch of 50 customers. Figure 4 reports its measured

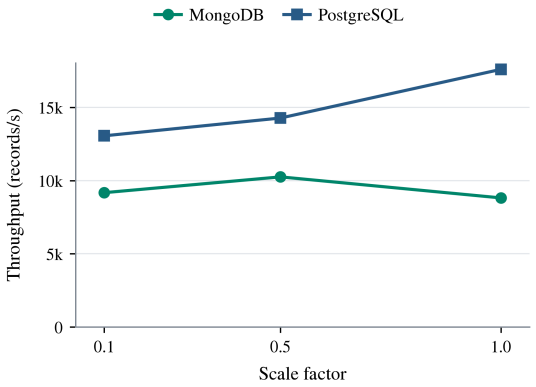


Figure 3: Median throughput of acknowledged 50-record insert batches with optional indexes enabled.

latency. The results do not identify a causal role for reclamation or checkpointing because these background activities were not profiled. At indexed SF 1.0, median delete-batch latency was 6.03 ms for MongoDB and 1.94 ms for PostgreSQL, with respective throughputs of 8,286 and 25,790 records/s and a reported 3.11× PostgreSQL speedup. Without optional indexes at the same scale, MongoDB recorded 208.05 ms and 240 records/s, compared with 13,173.75 ms and 3.8 records/s for PostgreSQL, a reported 63.32× MongoDB speedup.

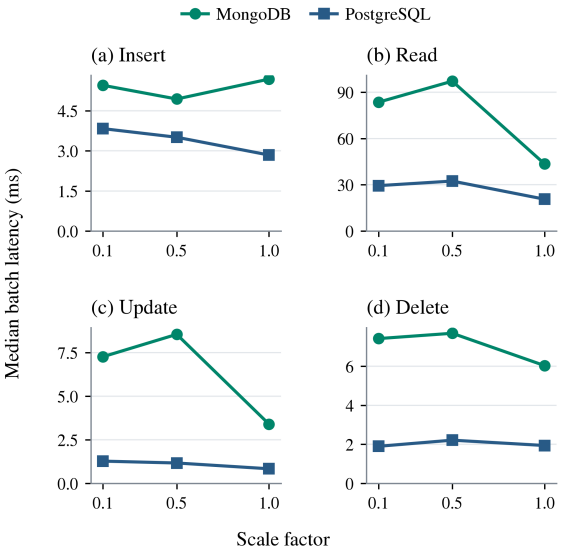


Figure 4: Median latency of indexed 50-record CRUD batches. Reads use successive key lookups. Each write trial begins from its restored state.

D. Query Response Time

We measured latency at the client as

$$L = t_{\text{end}} - t_{\text{start}} \tag{2}$$

where t_{start} is recorded immediately before query submission and t_{end} after receipt of the final result. Figure 5 reports the median of each complete ten-run configuration. Both database servers ran on the same Windows host over loopback connections. The results characterize the recorded single-node settings, derived datasets, and warm-cache workloads.

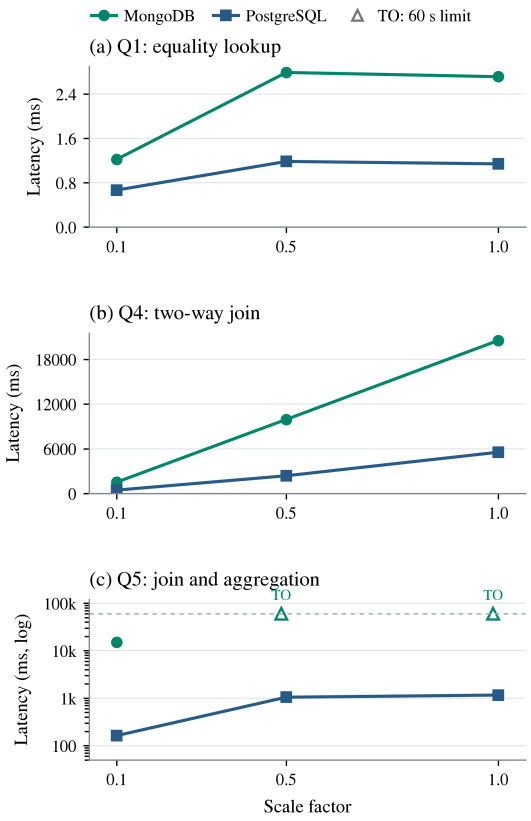


Figure 5: Median indexed-query latency by scale factor. TO marks the 60-second server limit and is not a measured median.

E. Storage Overhead

We measured the space allocated on disk to base data and B-tree indexes. The storage overhead ratio was

$$\rho = \frac{S_{\text{DB}}}{S_{\text{raw}}} \tag{3}$$

where S_{DB} is the total on-disk footprint (data plus indexes) and S_{raw} is the size of the original CSV dataset.

VII. ANALYSIS

A. Effect of Data Volume

Figure 6 reports measured $Q1$ medians. Figure 7 reports $Q2$ and $Q3$ with the fixed repository predicates. Their result counts are retained in the evidence, including zero-result cases. These observations do not constitute a controlled sweep of predicate selectivity.

1) Small ($SF = 0.1$): For indexed $Q1$, the median latency was 1.22 ms for MongoDB and 0.67 ms for PostgreSQL.

2) Moderate ($SF = 0.5$): For indexed $Q1$, the median latency was 2.79 ms for MongoDB and 1.18 ms for PostgreSQL.

3) Large ($SF = 1.0$): For indexed $Q1$, the median latency was 2.71 ms for MongoDB and 1.14 ms for PostgreSQL.

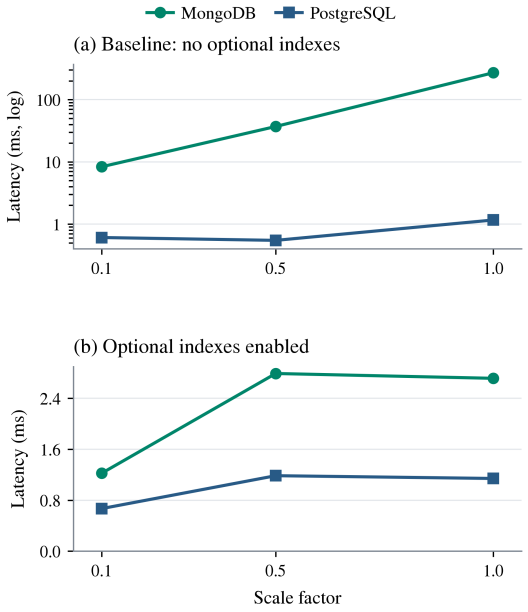


Figure 6: Measured equality-lookup medians by data volume and index configuration. The baseline retains PostgreSQL primary-key indexes and MongoDB identifier indexes.

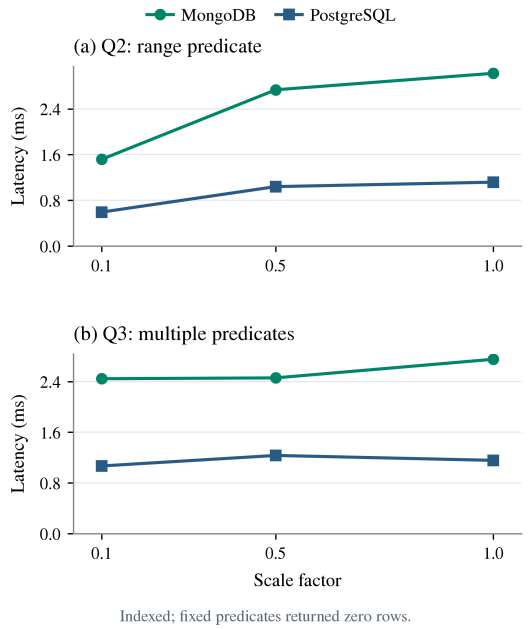


Figure 7: Indexed range and multi-condition query latency with the fixed repository predicates. Both queries returned zero rows in the recorded conditions.

B. Effect of Query Complexity

Figure 8 compares the measured $Q1$, $Q4$, and $Q5$ medians at SF 1.0. Figure 9 reports join and aggregation latency at all scales. Executed plans, rather than query syntax alone, identify the physical join strategies. The comparison does not assume that every MongoDB `$lookup` uses a nested loop or that PostgreSQL always selects a hash join.

1) Simple: An indexed equality lookup can use the relevant business-key index. PostgreSQL can also use its retained primary-key index in the baseline mode.

2) Moderate: The two-way join combines customers in the BUILDING market segment with their orders. Both engines return the same five projected fields. This avoids the larger nested payload returned by the original MongoDB pipeline.

3) Complex: Grouping and sorting may require intermediate state in both engines. The saved executed plans identify the chosen operators. Latency alone does not establish the memory cost or the reason for a difference between engines.

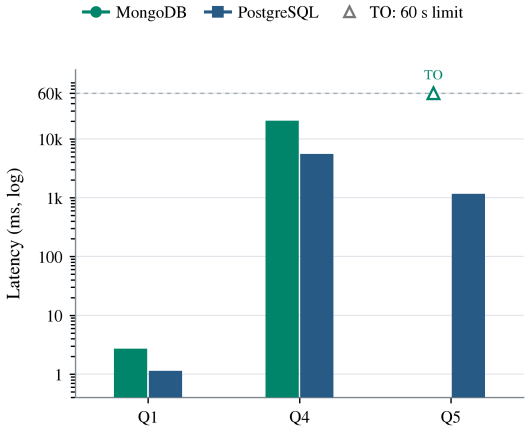


Figure 8: Median indexed-query latency at SF 1.0 on a logarithmic axis. TO marks the 60-second server limit.

C. Data Volume × Query Complexity Interaction

Table 3 retains the planned 3×6 matrix. 10 core-query comparisons completed with matching outputs. 10 of the 10 complete core-query comparisons had Holm-adjusted $p < 0.05$ across the full family of completed query comparisons. The ratio is $R = L_{\text{MongoDB}}/L_{\text{PostgreSQL}}$, computed from unrounded medians. The speedup divides the slower median by the faster one. Table p-values are unadjusted, two-sided Mann–Whitney results. Missing comparisons have no assigned winner or inferred p-value.

D. CRUD Performance

Figure 3 and Figure 4 report measured insert throughput and CRUD batch latency. The row counts are checked for every trial. Variability is summarized from actual repetitions, and no causal attribution to background

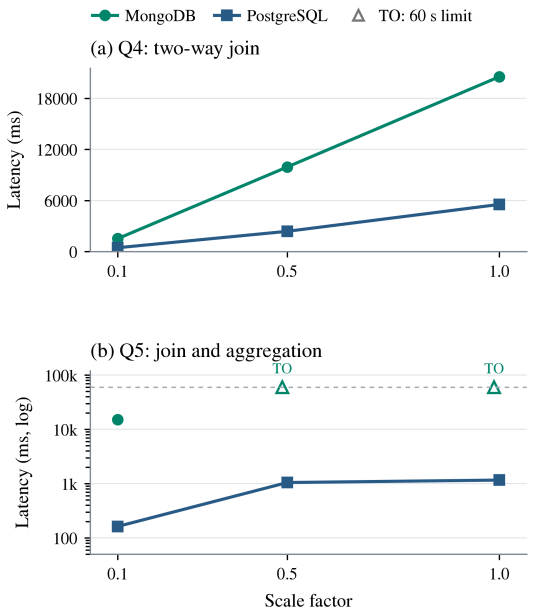


Figure 9: Median indexed join and aggregation latency across scale factors. TO marks the server limit; no median is inferred for a timeout.

storage-engine activity is made. For indexed SF 1.0, the 50-record key-read batch had median latency of 43.51 ms for MongoDB and 20.64 ms for PostgreSQL. The corresponding throughputs were 1,149 and 2,423 records/s, with a reported 2.11× PostgreSQL speedup.

E. Indexing Impact

Figure 10 compares measured read medians with and without the optional workload indexes. The change is calculated separately for each completed engine/workload configuration. Across complete pairs of index configurations, the observed unindexed-to-indexed median ratio ranged from 0.46 to 1583.76. A ratio above 1 indicates a lower indexed median, and a ratio below 1 indicates a higher indexed median. Table 4 and Figure 11 report executed-plan diagnostics. For the baseline equality lookup, PostgreSQL retains a business-key primary-key access path while MongoDB scans for a business key distinct from its mandatory identifier. This is an access-path difference within the tested schema, so it cannot establish an inherent point-lookup advantage for either storage model. Planning time is distinct from execution time. Shared-buffer hits count buffer accesses, rather than distinct disk blocks. Incomplete plans are marked NM.

F. Storage Analysis

Table 5 and Figure 12 use measured allocated data and index bytes. Raw size is the sum of the eight CSV file sizes. All displayed sizes use MiB (2^{20} bytes). PostgreSQL data allocation includes table and auxiliary storage, with indexes reported separately. MongoDB data allocation uses `storageSize`, and its indexes use `indexSize`. Uncompressed BSON `dataSize` is retained separately in the evidence. The storage ratio uses

Table 3
Master Latency & Performance-Ratio Matrix ($R < 1$: MongoDB faster; $R > 1$: PostgreSQL faster)

Phase	SF	Workload	Index Mode	MongoDB (ms)	PostgreSQL (ms)	Ratio (R)	Speedup	Winner	p-value
Small	0.1	$Q1$ (Equality)	none	8.34	0.61	13.72	13.72×	**PostgreSQL**	0.000183
Small	0.1	$Q1$ (Equality)	indexed	1.22	0.67	1.83	1.83×	**PostgreSQL**	0.00283
Small	0.1	$Q4$ (Join)	none	*TO*	366.33	–	–	–	–
Small	0.1	$Q4$ (Join)	indexed	1530.13	459.48	3.33	3.33×	**PostgreSQL**	0.000183
Small	0.1	$Q5$ (Aggregate)	none	*TO*	208.97	–	–	–	–
Small	0.1	$Q5$ (Aggregate)	indexed	15121.88	162.80	92.89	92.89×	**PostgreSQL**	0.000183
Medium	0.5	$Q1$ (Equality)	none	36.99	0.55	67.60	67.60×	**PostgreSQL**	0.000183
Medium	0.5	$Q1$ (Equality)	indexed	2.79	1.18	2.35	2.35×	**PostgreSQL**	0.000183
Medium	0.5	$Q4$ (Join)	none	*TO*	2126.19	–	–	–	–
Medium	0.5	$Q4$ (Join)	indexed	9937.65	2384.42	4.17	4.17×	**PostgreSQL**	0.000183
Medium	0.5	$Q5$ (Aggregate)	none	*TO*	925.41	–	–	–	–
Medium	0.5	$Q5$ (Aggregate)	indexed	*TO*	1049.07	–	–	–	–
Large	1.0	$Q1$ (Equality)	none	270.22	1.16	232.46	232.46×	**PostgreSQL**	0.000183
Large	1.0	$Q1$ (Equality)	indexed	2.71	1.14	2.38	2.38×	**PostgreSQL**	0.000246
Large	1.0	$Q4$ (Join)	none	*TO*	*TO*	–	–	–	–
Large	1.0	$Q4$ (Join)	indexed	20521.51	5538.78	3.71	3.71×	**PostgreSQL**	0.000183
Large	1.0	$Q5$ (Aggregate)	none	*TO*	1672.61	–	–	–	–
Large	1.0	$Q5$ (Aggregate)	indexed	*TO*	1163.76	–	–	–	–

Table 4
Query Execution-Plan Diagnostics: Physical Scan Types and Memory Block Counts

SF	Workload	Index Mode	MongoDB			PostgreSQL		
			Stage	Docs Examined	Keys Examined	Plan Node	Plan Time	Shared Hit
0.1	$Q1$	none	PROJECTION_SIMPLE / COLLSCAN	15000	0	Index Scan	0.079	3
0.1	$Q1$	indexed	PROJECTION_SIMPLE / FETCH / IXSCAN	1	1	Index Scan	0.157	3
0.1	$Q4$	none	NM	NM	NM	Hash Join	0.191	2316
0.1	$Q4$	indexed	PROJECTION_SIMPLE / FETCH / IXSCAN	32967	32967	Hash Join	0.418	2238
0.5	$Q1$	none	PROJECTION_SIMPLE / COLLSCAN	75000	0	Index Scan	0.122	3
0.5	$Q1$	indexed	PROJECTION_SIMPLE / FETCH / IXSCAN	1	1	Index Scan	0.195	0
1.0	$Q1$	none	PROJECTION_SIMPLE / COLLSCAN	150000	0	Index Scan	0.163	4
1.0	$Q1$	indexed	PROJECTION_SIMPLE / FETCH / IXSCAN	1	1	Index Scan	0.091	0
1.0	$Q5$	indexed	NM	NM	NM	Sort	0.369	13126



Figure 10: Median latency with and without optional indexes at SF 1.0, using identical logarithmic axes. TO marks the server limit.

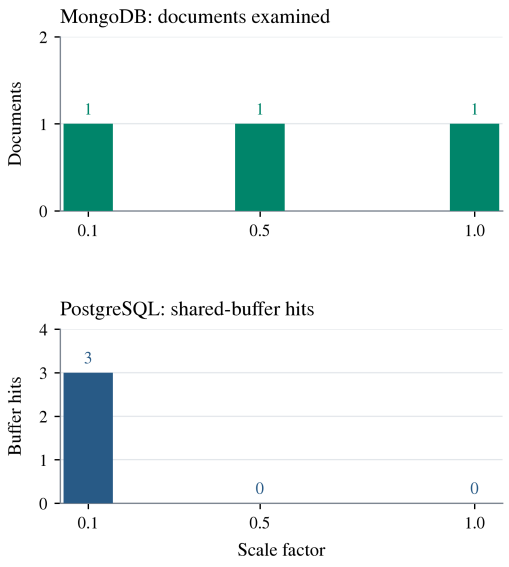


Figure 11: Indexed $Q1$ execution-plan counters. MongoDB documents examined and PostgreSQL shared-buffer hits have different meanings. Zero shared hits are reported as recorded.

allocated data plus indexes divided by raw CSV bytes. At SF 1.0 with optional indexes, the measured allocated-storage ratios were 1.00× for MongoDB and 1.68× for PostgreSQL. PostgreSQL WAL and MongoDB journal files are excluded from this table/collection-and-index comparison. Both servers were explicitly checkpointed before the storage snapshots.

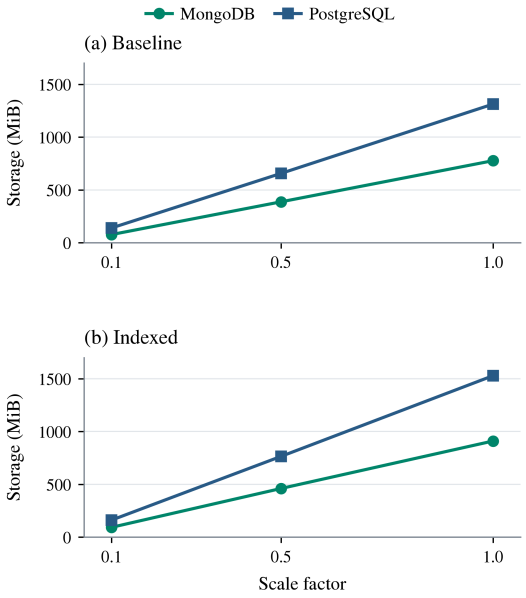


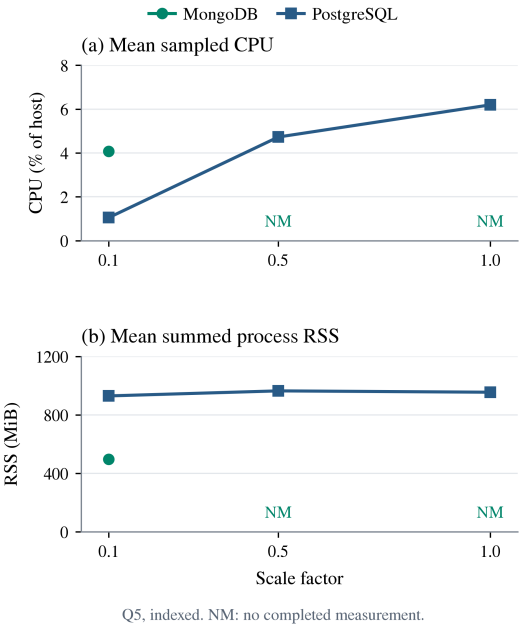
Figure 12: Measured allocated data plus index storage. Ratios use the corresponding exported or generated CSV byte totals.

G. Scalability and Resource Utilization

Table 6 and Figure 13 report sampled local server-process CPU and aggregate RSS. CPU sums per-process rates and normalizes by the 24 logical processors. Process RSS is sampled at 50-ms intervals and at query boundaries, with the arithmetic mean reported across runs. PostgreSQL includes the running instance's processes and other backends. Summed RSS may count shared pages more than once, and parallel workers created after enumeration may be missed. Short-query CPU rates are limited by operating-system accounting resolution. These measurements describe observed process footprints and do not establish an equivalent reduction in physical RAM requirements.

VIII. RESULTS AND DISCUSSION

10 of the 10 complete core-query comparisons had Holm-adjusted $p < 0.05$ across the full family of completed query comparisons. Both database servers ran on the same Windows host over loopback connections. The results characterize the recorded single-node settings, derived datasets, and warm-cache workloads. The executed plans provide operator-level context, but they do not independently establish causal explanations involving parsing, tuple construction, checkpointing, or materialization. Complete output-matched comparisons use the actual run distributions [30]. Tests on ten repeated



Q5, indexed. NM: no completed measurement.

Figure 13: Sampled CPU and summed process RSS for indexed *Q5*. NM denotes an unavailable complete measurement. RSS can double-count shared pages, and short-query CPU rates have limited resolution.

warm-cache runs describe these particular configurations. They do not establish universal engine rankings.

IX. WORKLOAD-BASED COMPARISON

Figure 14 summarizes the measured indexed-query latency efficiencies. For each available query at SF 0.1, the faster median is divided by each engine's median, so that the faster engine scores 1. Axes without complete output-matched measurements are excluded. Memory efficiency and schema flexibility receive no invented numeric scores.

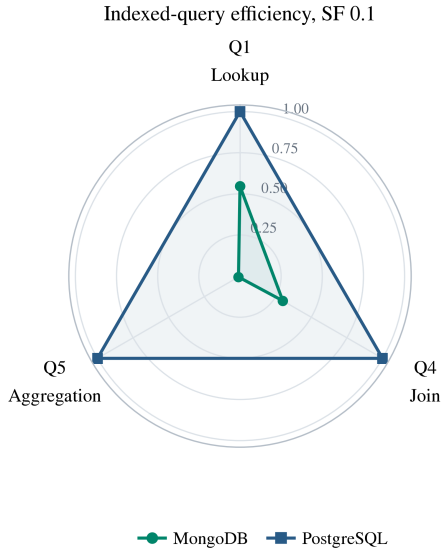


Figure 14: Indexed-query latency efficiency at SF 0.1. A score of 1 identifies the faster median for an output-validated comparison. Resource efficiency is unmeasured.

Table 5
Storage footprint and index overhead (MiB), relative to raw CSV size

Phase	SF	Raw CSV	Index Mode	MongoDB				PostgreSQL			
				Data	Index	Total	Ratio	Data	Index	Total	Ratio
Small	0.1	87.91	none	68.43	8.72	77.15	0.88×	113.12	26.69	139.80	1.59×
Small	0.1	87.91	indexed	68.43	24.71	93.14	1.06×	113.12	48.52	161.63	1.84×
Medium	0.5	449.36	none	342.42	44.86	387.27	0.86×	563.50	92.98	656.48	1.46×
Medium	0.5	449.36	indexed	342.42	118.61	461.02	1.03×	563.50	201.55	765.05	1.70×
Large	1.0	909.85	none	685.13	91.44	776.57	0.85×	1126.45	185.73	1312.18	1.44×
Large	1.0	909.85	indexed	685.13	225.13	910.26	1.00×	1126.45	402.11	1528.55	1.68×

Table 6
Sampled local server CPU (host-normalized) and summed process RSS (MiB)

Phase	SF	Workload	Index Mode	MongoDB			PostgreSQL		
				Avg CPU	Peak CPU	RAM (RSS)	Avg CPU	Peak CPU	RAM (RSS)
Small	0.1	$Q1$	none	1.45%	7.71%	481.21	0.00%	0.00%	871.07
Small	0.1	$Q4$	indexed	3.69%	6.14%	486.45	0.17%	0.46%	897.25
Small	0.1	$Q5$	indexed	4.07%	6.89%	497.22	1.05%	2.98%	929.70
Medium	0.5	$Q1$	none	3.75%	5.33%	1133.18	0.00%	0.00%	938.32
Medium	0.5	$Q4$	indexed	3.72%	6.78%	1146.80	0.89%	3.92%	956.90
Medium	0.5	$Q5$	indexed	*NM*	*NM*	*NM*	4.73%	11.71%	964.16
Large	1.0	$Q1$	none	3.97%	14.82%	1046.07	0.01%	0.13%	959.75
Large	1.0	$Q4$	indexed	3.59%	7.58%	1057.78	1.66%	5.40%	934.61
Large	1.0	$Q5$	indexed	*NM*	*NM*	*NM*	6.19%	16.28%	954.94

- **Choose MongoDB for:** document-oriented access and schema flexibility when these match application requirements. The measured local MongoDB timings in Table 3 determine its performance in the tested deployment.
- **Choose PostgreSQL for:** relational constraints and queries combining entities when these match application requirements. Its measured latency and storage are reported in the tables. The experiment does not quantify large-scale financial analytics or comparable engine-memory requirements.

CONCLUSION

The study attempted the planned comparison across three scale factors (0.1, 0.5, and 1.0), three core workloads ($Q1$, $Q4$, and $Q5$), and two optional-index configurations. PostgreSQL had the lower observed median in 10 of the 10 complete core-query comparisons, and MongoDB in 0. Completed-comparison speedups ranged from 1.83× to 232.46×. Both database servers ran on the same Windows host over loopback connections. The results characterize the recorded single-node settings, derived datasets, and warm-cache workloads. The evidence replaces formula-generated estimates with measured medians, recorded timeouts, and explicit unavailable values. Conclusions about H1–H5 require the corresponding volume, query, index, write, and storage measurements. A blanket confirmation of all hypotheses is not warranted by between-engine latency tests alone.

LIMITATIONS

Internal: Repeated warm-cache runs, operating-system activity, other host activity, and fixed engine settings can affect timings [24]. **External:** The experiment targets SF 0.1–1.0 and a single local MongoDB deployment. Conditions that cannot finish loading or executing are identified in the evidence. **Construct:** Client latency includes driver, serialization, result transfer, and loopback network costs. PostgreSQL NUMERIC and MongoDB double arithmetic differ. Aggregate process RSS may double-count shared pages, and CPU accounting is coarse for short queries. Baseline primary-key access paths differ between engines, and configured cache budgets are unequal. **Scope:** $Q2$ and $Q3$ and detailed CRUD measurements are supplied in the evidence and figures. The core latency table retains $Q1$, $Q4$, and $Q5$. Local query identifiers differ from the standard TPC-H query numbering.

FUTURE WORK

- Extend the analysis of $Q2$ and $Q3$ to a controlled range of nonempty predicate selectivities.
- Compare deeply nested or denormalized MongoDB schemas with the corresponding set of normalized PostgreSQL tables.
- Stress-test both engines on multi-terabyte datasets in clusters with 10 Gbps network links.
- Measure throughput at high concurrency using OLTP-Bench [23].
- Quantify the cost of distributed consensus under

simultaneous reads and writes.

REFERENCES

[1] M. Stonebraker and U. Çetintemel, "'One Size Fits All': An Idea Whose Time Has Come and Gone," in *Proc. Int. Conf. Data Engineering (ICDE)*, 2005, pp. 2–11.

[2] A. Pavlo and M. Aslett, "What's Really New with NewSQL?," *ACM SIGMOD Record*, vol. 45, no. 2, pp. 45–55, 2016.

[3] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[4] The PostgreSQL Global Development Group, "PostgreSQL 18 Documentation," accessed Nov. 13, 2026. [Online]. Available: `https://www.postgresql.org/docs/18/`

[5] The PostgreSQL Global Development Group, "PostgreSQL 18 EXPLAIN Documentation," accessed Nov. 13, 2026. [Online]. Available: `https://www.postgresql.org/docs/18/sql-explain.html`

[6] MongoDB, Inc., "MongoDB 8.0 Manual," accessed Nov. 13, 2026. [Online]. Available: `https://www.mongodb.com/docs/v8.0/`

[7] MongoDB, Inc., "Aggregation Pipeline — MongoDB 8.0 Manual," accessed Nov. 13, 2026. [Online]. Available: `https://www.mongodb.com/docs/v8.0/core/aggregation-pipeline/`

[8] MongoDB, Inc., "Explain Results — MongoDB 8.0 Manual," accessed Nov. 13, 2026. [Online]. Available: `https://www.mongodb.com/docs/v8.0/reference/explain-results/`

[9] R. Cattell, "Scalable SQL and NoSQL Data Stores," *ACM SIGMOD Record*, vol. 39, no. 4, pp. 12–27, 2011.

[10] M. Stonebraker *et al.*, "The End of an Architectural Era (It's Time for a Complete Rewrite)," in *Proc. 33rd Int. Conf. Very Large Data Bases (VLDB)*, 2007, pp. 1150–1160.

[11] P. J. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley, 2012.

[12] N. Leavitt, "Will NoSQL Databases Live Up to Their Promise?," *IEEE Computer*, vol. 43, no. 2, pp. 12–14, 2010.

[13] M. A. Mohamed, O. G. Altrafi, and M. O. Ismail, "Relational vs. NoSQL Databases: A Survey," *Int. J. Computer and Information Technology*, vol. 3, no. 3, pp. 598–601, 2014.

[14] J. Han *et al.*, "Survey on NoSQL Database," in *Proc. 6th Int. Conf. Pervasive Computing and Applications (ICPCA)*, 2011, pp. 363–366.

[15] F. Gessert, W. Wingerath, S. Friedrich, and N. Ritter, "NoSQL Database Systems: A Survey and Decision Guidance," *Computer Science — Research and Development*, vol. 32, no. 3–4, pp. 353–365, 2017.

[16] C. Győrödi *et al.*, "A Comparative Study: MongoDB vs. MySQL," in *Proc. 13th Int. Conf. Engineering of Modern Electric Systems (EMES)*, 2015, pp. 1–6.

[17] Z. Parker, S. Poe, and S. V. Vrbsky, "Comparing NoSQL MongoDB to an SQL DB," in *Proc. 51st ACM Southeast Conf. (ACMSE)*, 2013, pp. 1–6.

[18] M.-G. Jung *et al.*, "A Study on Data Input and Output Performance Comparison of MongoDB and PostgreSQL in the Big Data Environment," in *Proc. 8th Int. Conf. Database Theory and Application (DTA)*, 2015, pp. 14–17.

[19] A. Boicea, F. Radulescu, and L. I. Agapin, "MongoDB vs Oracle — Database Comparison," in *Proc. 3rd Int. Conf. Emerging Intelligent Data and Web Technologies (EIDWT)*, 2012, pp. 330–335.

[20] Y. Li and S. Manoharan, "A Performance Comparison of SQL and NoSQL Databases," in *IEEE PACRIM*, 2013, pp. 15–19.

[21] V. Abramova and J. Bernardino, "NoSQL Databases: MongoDB vs Cassandra," in *Proc. C3S2E*, 2013, pp. 14–22.

[22] B. F. Cooper *et al.*, "Benchmarking Cloud Serving Systems with YCSB," in *Proc. 1st ACM Symp. Cloud Computing (SoCC)*, 2010, pp. 143–154.

[23] D. E. Difallah, A. Pavlo, C. Curino, and P. Cudre-Mauroux, "OLTP-Bench: An Extensible Testbed for Benchmarking Relational Databases," *Proc. VLDB Endowment*, vol. 7, no. 4, pp. 277–288, 2013.

[24] M. Raasveldt *et al.*, "Fair Benchmarking Considered Difficult: Common Pitfalls in Database Performance Testing," in *Proc. DBTest Workshop*, 2018, pp. 1–6.

[25] C.-O. Truică *et al.*, "Performance Evaluation for CRUD Operations in Asynchronously Replicated Document Oriented Database," in *Proc. 20th CSCS*, 2015, pp. 191–196.

[26] S. Chaudhuri, "An Overview of Query Optimization in Relational Systems," in *Proc. ACM PODS*, 1998, pp. 34–43.

[27] G. Graefe, "Modern B-Tree Techniques," *Foundations and Trends in Databases*, vol. 3, no. 4, pp. 203–402, 2011.

[28] Transaction Processing Performance Council, "TPC Benchmark H (Decision Support) Standard Specification, Revision 3.0.1," 2022. [Online]. Available: `https://www.tpc.org/tpch/`

[29] M. Poess and C. Floyd, "New TPC Benchmarks for Decision Support and Web Commerce," *ACM SIGMOD Record*, vol. 29, no. 4, pp. 64–71, 2000.

[30] H. B. Mann and D. R. Whitney, "On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.

★ ★ ★